

Mini Transaction Ledger

Architecture, code structure, API internals and Docker setup

Abir Rahman

abirr7140@gmail.com | linkedin.com/in/abir-rahman-059169264 | github.com/abirzishan32

1 System overview

The application maintains a double entry ledger for a single authenticated user. Each user holds a private chart of accounts, posts transactions against those accounts, and retrieves balances, per account statements and a trial balance. No data is shared between users and the system defines no administrative role.

Double entry requires that every transaction affect exactly two accounts. Value leaves one account and arrives at the other, so a posting is recorded as two rows whose amounts sum to zero. The schema declares no separate debit and credit columns. A single signed `Amount` carries both senses: a positive value is a debit and a negative value is a credit. A consulting fee of 250.00 received into Cash is stored as +250.00 against Cash and -250.00 against Consulting Income.

Balances are not stored. The balance of an account is defined as `SUM(Amount)` over the entries that reference it, evaluated when the value is requested. A stored balance would constitute a second representation of a fact the entries already hold, and two representations of one fact diverge over time. Deriving the balance removes that class of defect entirely.

2 Application architecture

The system has four parts: the browser, the frontend, which is a Next.js server that renders pages and holds the session, the backend, which is an ASP.NET Core API that owns every rule, and the database. Figure 1 states the components of each and the traffic that passes between them.

2.1 Separation of the frontend and the backend

The browser does not communicate with the ASP.NET Core API. It communicates with a Next.js server, which issues the corresponding request to the API on the user's behalf. This arrangement is known as a backend for frontend.

The determining factor is the access token. A browser calling the API directly would have to hold a JSON Web Token, and every storage location a browser offers is readable by any script on the page: `localStorage`, `sessionStorage` and cookies without the `HttpOnly` attribute are all exposed to a cross site scripting defect. The Next.js server therefore holds both tokens as `HttpOnly` cookies, which the browser transmits automatically but never exposes to JavaScript. The token is read from the cookie in `lib/auth.ts` and attached to the outbound request in `lib/api.ts`, and is never serialised into markup.

Two further properties follow from this separation. The address of the API is held in a server side environment variable, `API_BASE_URL`, which carries no `NEXT_PUBLIC_` prefix and is consequently not addressable from a browser. In addition, `lib/api.ts` and `lib/auth.ts` both import `server-only` on their first line, which converts any attempt to import them from a client component into a compilation failure rather than a runtime disclosure.

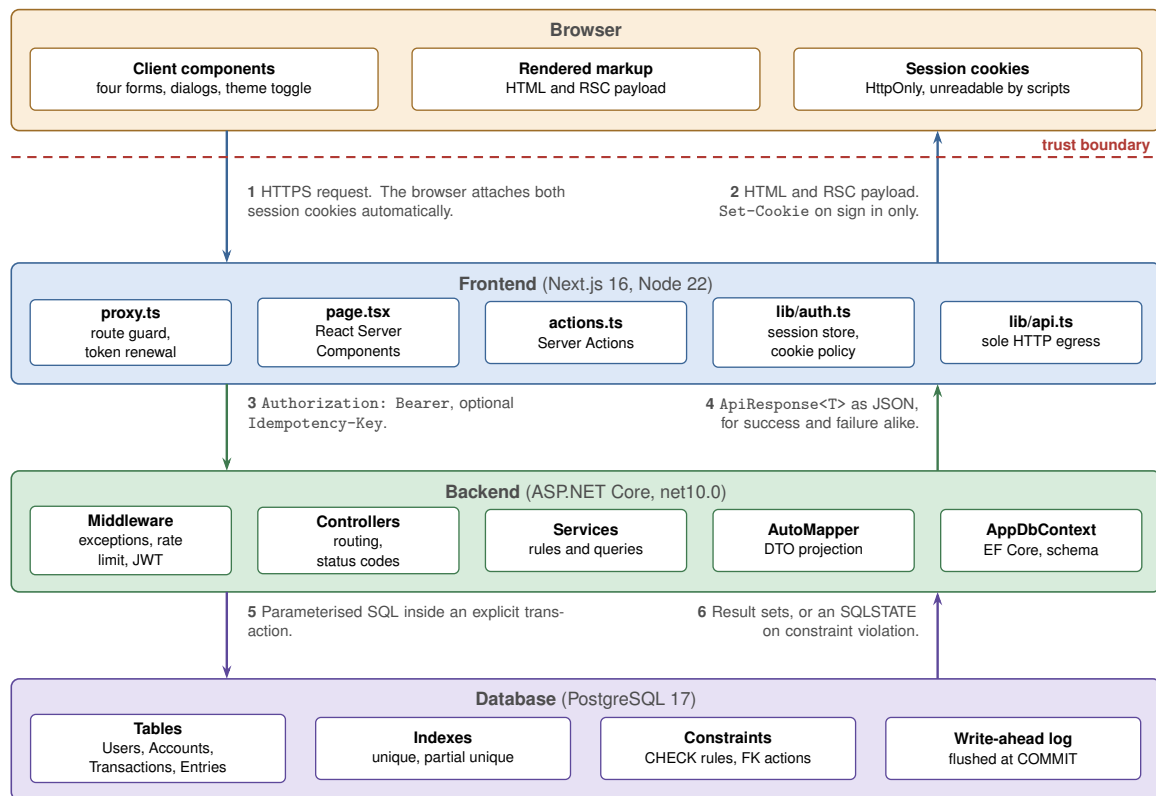


Figure 1: The four parts of the system and the traffic between them. No access token, signing key or database address exists above the trust boundary.

2.2 Distribution of responsibility

Each part holds a distinct responsibility and none duplicates the work of another. Table 1 states the allocation.

Table 1: Allocation of responsibility across the four parts of the system.

Component	Responsibility
Browser	Collects input and renders markup produced elsewhere. Holds no credential in any readable form and performs no data fetch of its own.
Frontend	Renders pages, executes Server Actions, holds the session, and performs the only outbound HTTP calls to the API. Enforces no business rule.
Backend	Authenticates every request, validates every request body, and owns all business rules, all queries and all transaction boundaries.
Database	Stores the ledger and enforces the invariants that must hold irrespective of the code path that reaches the table.

Next.js renders most pages as React Server Components. A page such as `trial-balance/page.tsx` is an `async` function that awaits its own data and returns completed markup. The browser receives the result rather than the request that produced it, so the initial render involves no client side fetch and exposes no API call to the network inspector.

Client components are used only where the browser must perform work of its own: the four forms, the password visibility control, the theme provider with its toggle, and the sidebar. These are marked `"use client"` and none of them performs a fetch. Form submission invokes a Server Action, a function marked `"use server"` that executes on the Next.js server and receives the form data. `postTransaction` in `app/dashboard/transactions/actions.ts` is one such function.

2.3 Session continuity and token renewal

`proxy.ts` executes before every request matching `/dashboard/:path*`, `/login` or `/register`. It performs three functions: it redirects unauthenticated visitors away from the dashboard, redirects authenticated visitors away from the authentication pages, and renews an access token that falls within 30 seconds of expiry so that the page beneath it never receives an expired token. It constitutes a convenience layer rather than a security boundary, since it executes only on the paths named in its matcher. Every page and every action therefore repeats the session check independently.

Both cookies carry a seven day `maxAge`, which deliberately outlives the 15 minute access token. Renewal reads the user identifier from the `sub` claim of the expired access token, so discarding that cookie at expiry would leave the refresh token unusable.

After a renewal, `proxy.ts` rewrites the inbound `Cookie` header in addition to setting the outbound one:

```
request.cookies.set(ACCESS_TOKEN_COOKIE, refreshed.data.accessToken);
const headers = new Headers(request.headers);
headers.set("cookie", request.cookies.toString());
const response = NextResponse.next({ request: { headers } });
```

A redirect would be simpler and incorrect. A redirect converts a Server Action POST into a GET, which discards the submission without reporting a failure. The consequence would be a transaction that the user submitted, that was never recorded, and about which no error was raised.

2.4 Statelessness and deployment topology

Neither server retains state between requests. The API derives the caller's identity from the token presented with each request and stores nothing about the access token it issued, and the Next.js server holds the session in the client's cookies rather than in memory. Both servers can therefore be replicated behind a load balancer without sticky sessions, leaving PostgreSQL as the only stateful component. The refresh token digest is the sole piece of authentication state, and it is a row rather than process memory.

3 Data model

The schema comprises four tables. `Users` holds credentials, `Accounts` holds the chart of accounts belonging to one user, `Transactions` holds the header of a posting, and `Entries` holds the two sides of that posting. Figure 2 states the keys, the cardinalities and the referential actions.

3.1 Schema design decisions

`Amount` is declared `numeric(18,2)` rather than a floating point type. Binary floating point cannot represent 0.1 exactly, and in a ledger the resulting error accumulates across postings until the trial balance ceases to agree with itself. A check constraint additionally rejects an amount of zero, since an entry of zero records no economic event.

`Username`, `Email` and `Account.Name` use the PostgreSQL `citext` type, which compares case insensitively. Placing this rule in the database rather than in application code ensures that "Cash" and "cash" collide regardless of which code path reaches the table. The length limits that `varchar` would otherwise have supplied are retained as check constraints.

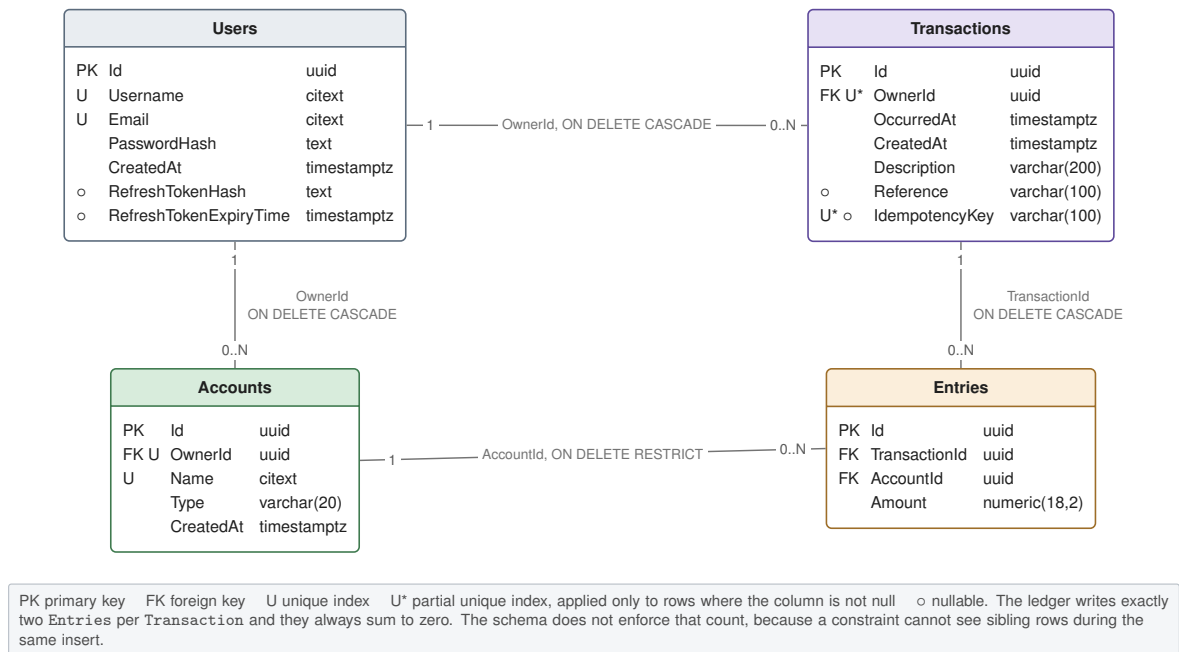


Figure 2: The four tables, their keys and their delete rules. Balances are absent by design: an account balance is `SUM(Amount)` over its entries.

The two referential actions differ deliberately. Deletion of a transaction cascades to its entries, because an entry without its header carries no meaning. Deletion of an account that has been posted to is refused by `ON DELETE RESTRICT`, because such a deletion would destroy history and unbalance every transaction in which the account appears.

`Account.Type` is stored by name rather than by ordinal, through `HasConversion<string>()` in the model and `JsonStringEnumConverter` on the wire. Reordering the `AccountType` enumeration would otherwise reinterpret every stored row and every value already sent to a client.

Three indexes serve a functional purpose rather than a performance one. The unique index on `(OwnerId, Name)` prevents two concurrent requests from creating the same account twice. The partial unique index on `(OwnerId, IdempotencyKey)` performs the equivalent function for duplicate postings, and is partial so that the many transactions carrying no key do not collide with one another. The composite index on `(OwnerId, OccurredAt, CreatedAt)` matches the ordering used by every listing.

4 Principal code components

4.1 Backend

The backend is organised into four layers, each holding a single responsibility. A request enters at a controller, which reads the caller's identity and determines the response shape. Controllers issue no queries. A service owns the business rules and the queries. `AppDbContext` owns the schema. The data transfer objects own validation. Table 2 lists the files of the backend.

Table 2: Principal files of the backend and their responsibilities.

File	Responsibility
<code>Program.cs</code>	Service registration and middleware ordering. Applies pending migrations at startup, configures JWT validation with <code>ClockSkew</code> set to zero, and installs the rate limiter and the global exception handler.
<code>Controllers/</code>	Six endpoints for accounts, three for transactions and five for authentication. Each reads <code>User.GetUserId()</code> from the validated token and delegates to a service.

Table 2: Principal files of the backend and their responsibilities. (continued)

File	Responsibility
<code>AuthService.cs</code>	Registration, sign in, renewal and sign out. Owns BCrypt hashing, refresh token generation and rotation, and access token construction.
<code>AccountService.cs</code>	The chart of accounts, per account statements with a running balance, and the trial balance. Balances are produced by a correlated subquery in <code>WithBalance</code> .
<code>TransactionService.cs</code>	The write path: idempotency resolution, the account ownership check, the overdraft check with its row lock, and the single batched save.
<code>Data/AppDbContext.cs</code>	The complete schema, comprising indexes, check constraints, referential actions and the <code>citext</code> extension. EF Core derives the migrations from it.
<code>DTOs/</code>	Request and response shapes. The write objects carry the validation attributes and, where a rule spans more than one field, an <code>IValidatableObject</code> method.
<code>ValidateModelAttribute.cs</code>	Registered globally, so that no controller repeats the model state check and no controller can omit it.
<code>Controllers/ApiResponse.cs</code>	The response envelope returned by every endpoint, for success and failure alike.

Two helpers merit specific mention. `ConfigurationExtensions.Require` raises an exception when a configuration key is absent, so a deployment lacking a signing key or a connection string terminates at startup and names the missing value, rather than failing later on its first query or issuing tokens signed with an empty key. `ClaimsPrincipalExtensions.GetUserId` reads the subject claim from the validated token and is the sole means by which a service learns who is calling.

4.2 Frontend

Table 3 lists the files of the frontend.

Table 3: Principal files of the frontend and their responsibilities.

File	Responsibility
<code>proxy.ts</code>	Executes before matched requests. Redirects according to session state and renews an access token approaching expiry, rewriting the request's own cookie header so that the render below it uses the new value.
<code>lib/auth.ts</code>	Cookie names and attributes, session read and write, and decoding of the access token claims. Marked <code>server-only</code> .
<code>lib/api.ts</code>	The sole location from which an HTTP call to the API is issued. <code>apiFetch</code> does not raise and always returns the envelope. <code>apiFetchAuthenticated</code> attaches the caller's own token and accepts no user identifier parameter.
<code>lib/ledger.ts</code>	Typed wrappers for the five read endpoints, together with the TypeScript counterparts of the API data transfer objects.
<code>app/(auth)/actions.ts</code>	The <code>login</code> , <code>register</code> and <code>signOut</code> Server Actions.
<code>app/dashboard/*/actions.ts</code>	<code>createAccount</code> and <code>postTransaction</code> . Both invoke <code>revalidatePath</code> on success, so that the pages affected by a write are rebuilt.
<code>app/dashboard/*/page.tsx</code>	Server Components. Each awaits its data and returns completed markup.

Table 3: Principal files of the frontend and their responsibilities. (continued)

File	Responsibility
lib/use-dialog.ts	Dialog open state that closes only once the server confirms the submission, so that a rejected submission retains every value the user entered.

The types in `lib/ledger.ts` are written by hand rather than generated. Five small types impose a lower reading cost than a code generation step, and a divergence from the API surfaces as a type error at the point of use.

5 Internal operation of the API

5.1 Request processing pipeline

Every request traverses the same ordered sequence of middleware before a controller receives it. The order is declared in `Program.cs` and is significant: each stage may terminate the request, and a stage placed too late cannot protect the stages above it. Figure 3 states the sequence together with the function of each stage.

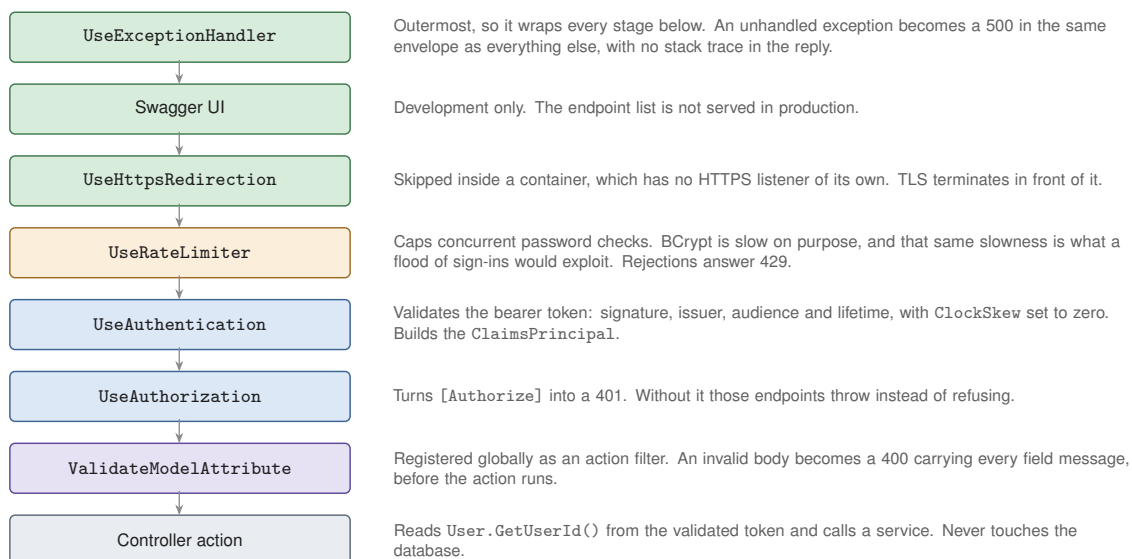


Figure 3: The middleware order in `Program.cs`. A request enters at the top; anything that fails is answered from the stage that caught it.

5.2 Uniform response envelope

Every endpoint returns `ApiResponse<T>` for both success and failure. The envelope carries `success`, `message`, `data`, `errors`, `statusCode` and `timestamp`. Validation failures use it, the rate limiter uses it, and the global exception handler uses it.

The consequence on the frontend is substantial. `apiFetch` in `lib/api.ts` does not raise. An unreachable API is caught and converted into the same envelope with status 503, and a response body that is not an envelope is converted likewise. Every caller therefore requires exactly one branch:

```
const result = await apiFetchAuthed<Transaction>("/api/transactions", { ... });
if (!result.success || !result.data) { return { message: result.message, ... }; }
```

No second error path and no `try/catch` block is required at any call site. The framework's automatic 400 response is disabled in `Program.cs` for the same reason, since it returns a `ProblemDetails` document that does not conform to this shape.

5.3 Request validation

Validation is declared on the data transfer objects and nowhere else. Rules governing a single field are expressed as attributes. Rules that read more than one field are expressed in an

`IValidatableObject` method, because no single attribute can span two properties. `TransactionWriteDto` declares two such rules: the debit and credit accounts must differ, and the date may not exceed the current time by more than one day.

The frontend does not restate these rules. The forms carry `noValidate` and the actions transmit the values as entered. The messages produced by the API are presented to the user unaltered. Two copies of a validation rule diverge over time, at which point the copy the user sees is no longer the copy that governs the outcome.

One field is deliberately exempt from normalisation. `Username`, `Email`, `Description` and `Reference` are trimmed in their property setters. `Password` is not. It is supplied to `BCrypt.HashPassword` and `BCrypt.Verify` exactly as the user entered it, including any leading or trailing whitespace, because trimming would produce a stored credential that the user cannot reproduce.

Dates are normalised in the service rather than in the data transfer object. `AsUtc` converts a local value and labels an unspecified one as UTC rather than converting it, because the form submits midnight UTC for a calendar day and a conversion would move the posting to the adjacent day.

5.4 Authentication and session management

Sign in returns two tokens with distinct functions. The access token is a JSON Web Token signed with HMAC SHA512, valid for 15 minutes, carrying the user identifier, the username and a unique token identifier. The API stores no record of it. Its signature constitutes the proof of issuance, and validating that signature is a local computation requiring no database access.

The refresh token consists of 32 bytes drawn from the cryptographic random number generator. It carries no information, and only a SHA256 digest of it is stored on the user row alongside an expiry seven days forward. It is rotated on every use.

The two are hashed by different algorithms for a specific reason. A password is hashed with `BCrypt`, which is deliberately slow, because a human selected password is drawn from a small set of probable values and the cost of each attempt is what renders exhaustive search impractical. A refresh token consists of 256 random bits with no probable values, so a deliberately slow hash would provide no additional resistance while imposing a cost on every renewal. SHA256 is therefore the appropriate choice.

Sign out clears the stored refresh token digest on the server and subsequently deletes the cookies. Clearing the digest is the decisive step, because renewal compares an incoming token against that stored value. Once it is null, every copy of that refresh token is invalid, not only the copy held by the browser that issued the request.

The stored digest is compared using `CryptographicOperations.FixedTimeEquals` rather than an ordinary string comparison. An ordinary comparison returns at the first differing byte, and that timing difference would allow the correct value to be recovered one byte at a time.

Figure 4 traces the sign in sequence from the form submission to the rendered dashboard.

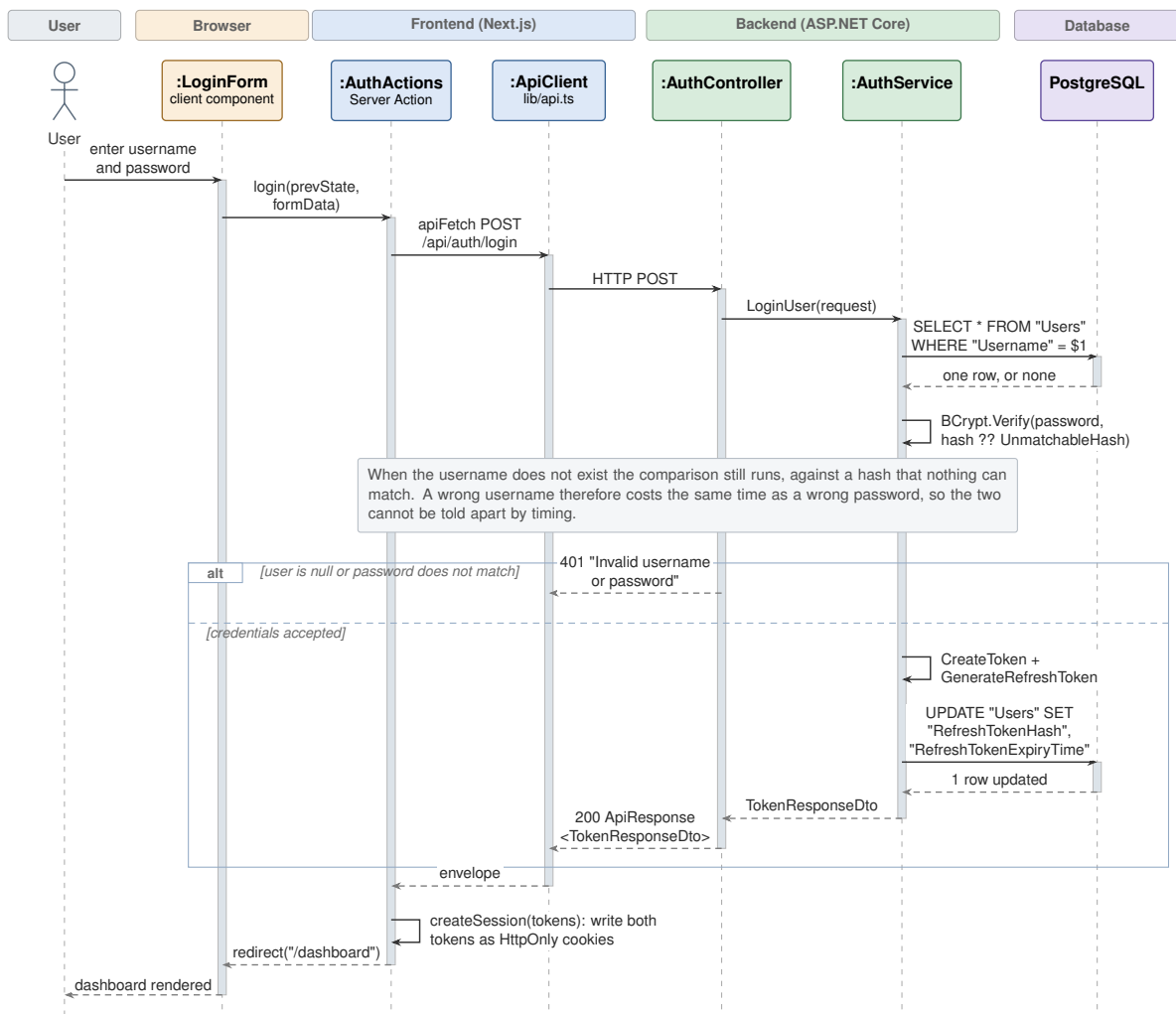


Figure 4: Signing in. Both tokens stop at the Next.js server and become HttpOnly cookies. The browser receives a rendered dashboard and never holds a credential.

5.5 Authorization and query scoping

Authentication establishes who is calling. Authorization restricts what that caller can reach, and in this system it is enforced by the way queries are constructed rather than by a check applied to their results. Both services declare a single scoping method and every read begins from it:

```
private IQueryable<Transaction> OwnedBy(Guid ownerId) =>
    _appDbContext.Transactions.Where(transaction => transaction.OwnerId == ownerId);
```

The owner identifier is supplied by `User.GetUserId()`, which reads the subject claim of the validated token. No endpoint accepts a user identifier as a parameter, so no caller can express a request on another user's behalf. The consequence is that a query returning another user's rows cannot be written without first removing the scoping method, rather than merely forgetting a check. Writes are scoped by the same value, which the service stamps on the row instead of accepting it from the request body.

A resource belonging to another user is reported as absent rather than forbidden. `GetAccount` returns null for both conditions and the controller answers 404 in either case, so the response cannot be used to determine which identifiers exist.

5.6 Recording a transaction

This write path is the only one whose correctness depends on the order in which its operations are performed. The `CreateTransaction` method of `TransactionService` performs five operations in a fixed sequence, which Figure 5 sets out in full.

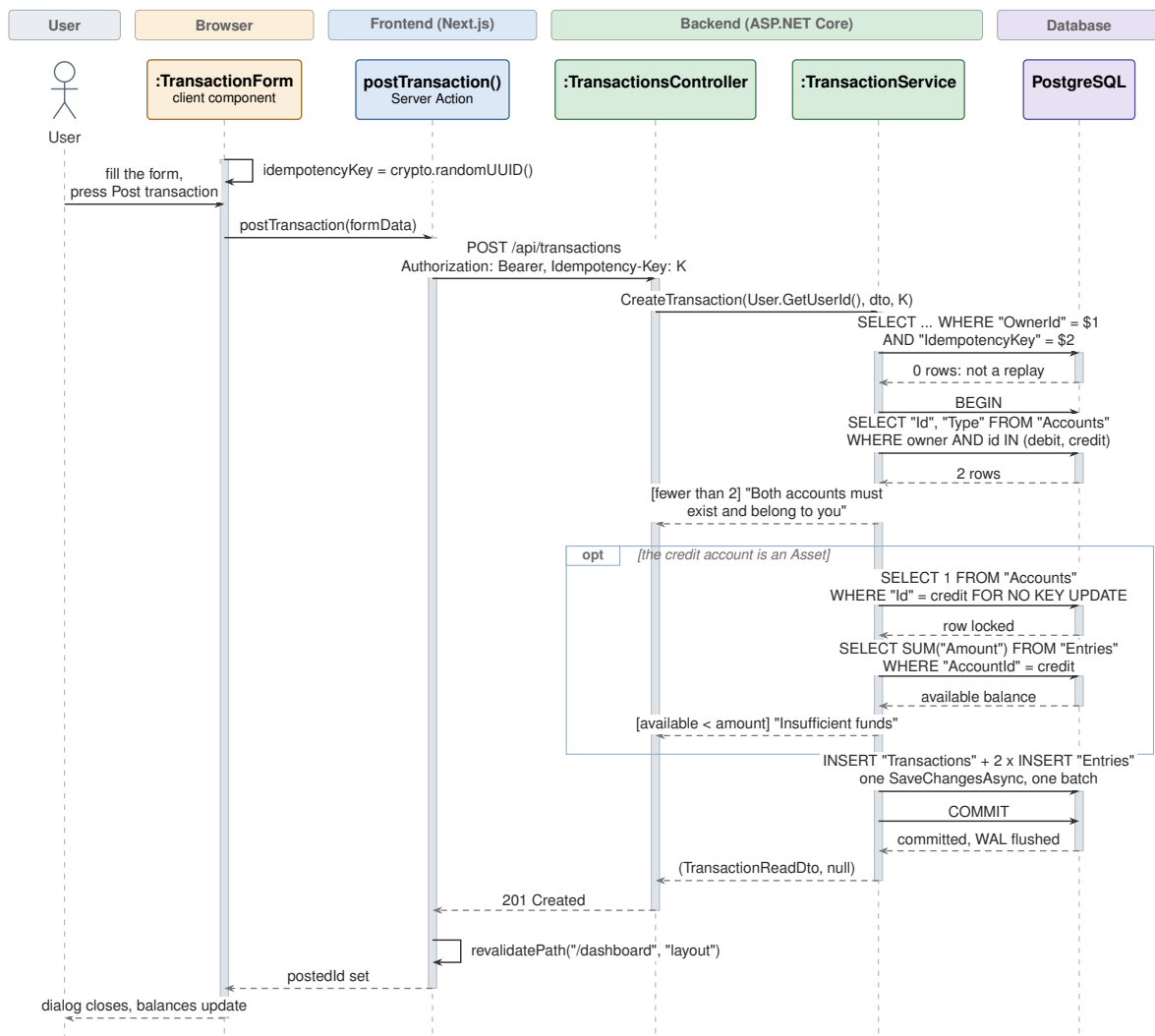


Figure 5: Posting a transaction. The idempotency key is settled before any lock is taken, and the header and both entries are written by a single `SaveChangesAsync` inside one database transaction.

The idempotency key is resolved first, before the database transaction is opened. A replayed request should incur no cost and must not queue behind an active posting against the same account.

The account lookup serves two purposes in a single query. It confirms that both accounts exist, and because the query filters on `OwnerId` it confirms that both belong to the caller. A result of fewer than two rows indicates that at least one account is absent or owned by another user. Both conditions produce an identical message, so the response cannot be used to determine which account identifiers exist.

The overdraft check executes only when the credit account is of type `Asset`. That restriction is deliberate. An `Asset` is the only account type for which a negative balance indicates an error. Liability, Income and Equity accounts are credit normal and hold negative balances in the ordinary course, so a floor applied to them would reject correct bookkeeping.

The header and both entries are written by one `SaveChangesAsync` call inside one database transaction. Section 5.10 sets out the atomicity that this provides.

5.7 Reading the ledger

Reads are projected rather than materialised. Each query ends in `ProjectTo`, which compiles the AutoMapper configuration into the SQL select list, so only the columns the data transfer object declares are retrieved and no entity is tracked by the change tracker. Balances are produced by the correlated subquery in `WithBalance`, which keeps a listing of any length to one statement rather than one statement per account.

Transaction history is paged by the API rather than by the client. The page number and size are bounded by `Range` attributes, the size to 100, and the count is taken before the slice so that the total describes the whole result rather than the page returned. `TotalPages` is derived from the count and the size rather than stored, so the two cannot disagree. The offset is computed as a 64 bit value:

```
var skip = (long)(page - 1) * pageSize;
if (skip >= totalCount) { return new PaginatedResult<TransactionReadDto> { ... }; }
```

The running balance on an account statement is accumulated in application code over the ordered projection rather than computed by a window function. The order is fixed by `OccurredAt` and then `CreatedAt`, the same order the composite index provides, so the accumulation follows the sequence the rows are already returned in.

5.8 Idempotent submission

An HTTP client that receives no response cannot determine what occurred. The request may have been lost in transit, or the response may have been lost after the transaction was committed. Retransmission is the only available remedy, and in the absence of protection a retransmission records the posting a second time.

The browser generates a UUID when the form mounts and transmits it in the `Idempotency-Key` header. The key is replaced only after a confirmed success, which constitutes the entire mechanism:

```
useEffect(() => { setIdempotencyKey(crypto.randomUUID()); }, [state.postedId]);
```

`state.postedId` changes only when the API confirms a posting. A failed attempt leaves it unchanged, the effect does not re-execute, and the retransmission carries the same key. Generating a new key on failure would convert every retransmission into a second genuine transaction.

The API examines the key before writing. That examination does not itself constitute the guarantee. Two requests carrying the same key may both complete the examination before either has committed, and both will find no existing row.

The losing request rolls back before reading. This ordering is required: after a failed statement PostgreSQL marks the transaction as aborted and rejects any subsequent query on it until the rollback is issued. The sequence is therefore rollback, re-read, and return the row written by the winner. Both callers receive the same transaction and exactly one row exists.

Figure 6 sets out the resulting interleaving.

5.9 Row level locking

Derived balances remove most contention from the system. A posting does not need to read the current balance before deciding what to write. Therefore, concurrent postings can simply append their entries, and the final balance remains correct regardless of which transaction is considered first.

The overdraft rule is different. It must first read the balance, check whether the withdrawal is allowed, and then write the new entries. Between these steps, another transaction may change the balance. For example, if an account has 100 and two concurrent withdrawals of 60 both read the balance as 100, both may pass the overdraft check and commit. The final balance then becomes -20. This is a form of write skew that can occur under Read Committed isolation.

To prevent this, the credit account row is locked before checking its balance. The second transaction must wait until the first transaction finishes. Once it obtains the lock, it reads the updated balance of 40 instead of 100 and therefore rejects the second withdrawal. The lock is acquired inside the same database transaction as the entry insertion, so it remains held until the transaction commits.

The lock mode used is `FOR NO KEY UPDATE` instead of `FOR UPDATE`. This is important because inserting an `Entry` also causes PostgreSQL to acquire an implicit `FOR KEY SHARE` lock on the referenced `Accounts` row. This lock ensures that the referenced account cannot be deleted or have its key changed while the entry is being inserted. As a result, every posting involves both account rows: one through the explicit lock and the other through the foreign key.

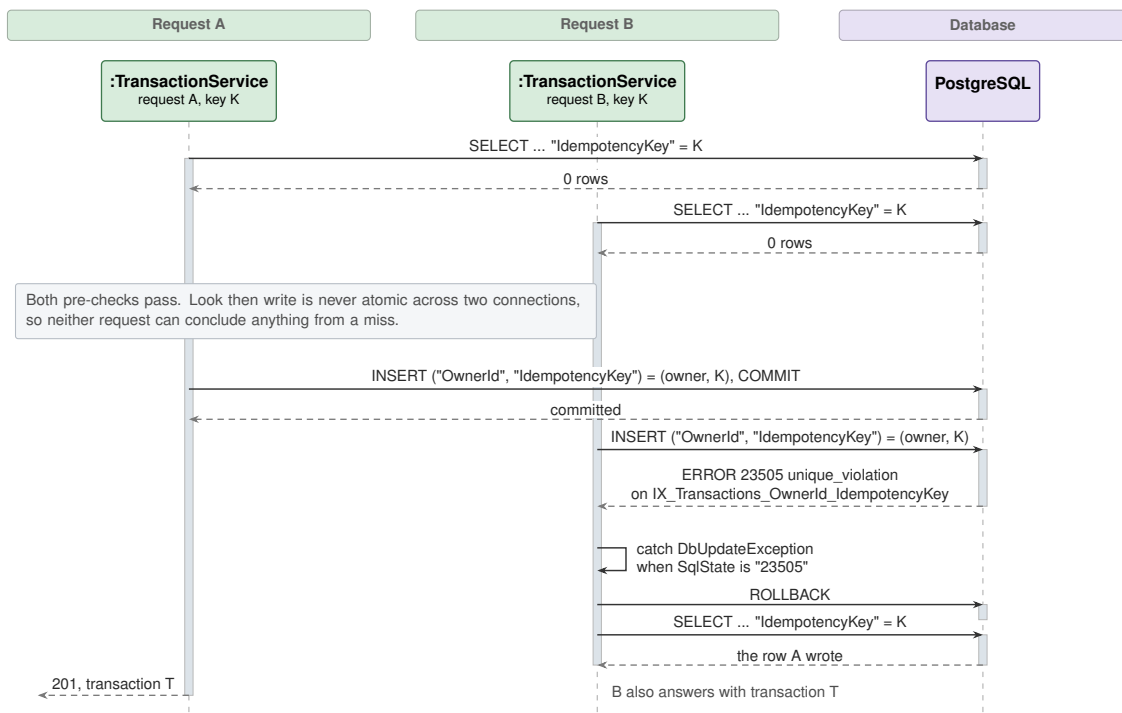


Figure 6: Two requests carrying one idempotency key. The partial unique index is the guarantee; PostgreSQL evaluates it inside the commit, where nothing can slip between the look and the write.

The choice of lock mode also matters for transfers in opposite directions. Suppose one transaction transfers money from A to B while another transfers money from B to A. With `FOR UPDATE`, each transaction explicitly locks one account and then requires a `FOR KEY SHARE` lock on the other account through the foreign key. These locks can conflict with the lock already held by the other transaction, causing both transactions to wait for each other.

With `FOR NO KEY UPDATE`, the situation is different. The implicit `FOR KEY SHARE` lock is compatible with the explicit lock, so the foreign key check does not create this conflict. At the same time, two transactions trying to acquire `FOR NO KEY UPDATE` on the same account still conflict with each other and therefore wait in sequence. This provides the required serialization for concurrent withdrawals without introducing the unnecessary conflict between transfers in opposite directions.

5.10 Transactional guarantees and concurrency control

The ledger depends on the four transactional properties commonly abbreviated ACID. This section states how each is obtained and which component is responsible for it.

Atomicity

A posting consists of three rows: one `Transactions` row and two `Entries` rows. `CreateTransaction` opens an explicit database transaction with `BeginTransactionAsync` before any of them is constructed, `EF Core` emits all three inserts from one `SaveChangesAsync` call as a single batch within that transaction, and `CommitAsync` makes them visible together. No concurrent reader can observe a debit without its credit, because no such state is ever committed.

Every exit path discards the work. The transaction is declared with `await using`, so disposal occurs on every return, and disposing an uncommitted transaction rolls it back. The ownership failure and the insufficient funds refusal both return early and are covered by this. The duplicate key case is handled explicitly, because the subsequent read must occur on a usable connection:

```

await using var dbTransaction = await _appDbContext.Database.BeginTransactionAsync();
...
try { await _appDbContext.SaveChangesAsync(); await dbTransaction.CommitAsync(); }
catch (DbUpdateException ex) when (key != null
    && ex.InnerException is PostgresException { SqlState: "23505" })
{
    await dbTransaction.RollbackAsync();
    return (await FindByIdempotencyKey(ownerId, key), null);
}
  
```

Consistency

Consistency requires that a committed transaction leave the database satisfying every declared invariant. The invariants are declared in `AppDbContext` and enforced by PostgreSQL, so they hold irrespective of the code path that reaches the table. They are the check constraints, unique indexes and referential actions set out in Section 3.1.

One invariant is established by construction rather than by the schema. The two entries of a posting are created as `+amount` and `-amount` within one object graph, so their sum is necessarily zero. No check constraint can express this, because a constraint is evaluated against a single row while the invariant spans two rows written in the same batch. The trial balance aggregates the same quantity across every account, so the equality of its totals verifies the invariant continuously rather than by a separate calculation.

The overdraft rule is an application level invariant rather than a schema level one, for the same reason: it constrains an aggregate over many rows, which no row constraint can evaluate.

Isolation

The database operates at PostgreSQL's default isolation level, Read Committed. Each statement observes a snapshot taken when that statement began, so no statement reads uncommitted data and readers are never blocked by writers.

The choice has a direct consequence for reporting. `GetTrialBalance` delegates to `GetAccounts`, which projects every balance as a correlated subquery within one `SELECT`. The whole trial balance is therefore computed from a single statement and a single snapshot, and cannot observe a posting in a partially applied state, even though the isolation level would permit two separate statements to disagree.

Read Committed permits non-repeatable reads, phantom rows and write skew. Of these, only write skew affects correctness here, and it arises on exactly one code path, which the row lock of Section 5.7 covers. The stronger levels were considered and rejected on the following grounds.

Serializable would remove the write skew without an explicit lock, but it detects conflicts rather than preventing them: it aborts one transaction with `SQLSTATE 40001` at commit time, after the work has been performed, which obliges every caller to implement retry logic. Repeatable Read imposes the same requirement and does not prevent write skew in any case, and optimistic concurrency using a version column likewise discovers the conflict only after the work is complete. Read Committed with one row lock on the single path that reads before it writes is the narrower arrangement.

Durability

`COMMIT` returns only after the corresponding write ahead log record has been flushed to durable storage. PostgreSQL enables `synchronous_commit` by default, so a transaction reported as committed survives an immediate loss of power to the database process.

Container durability is provided by the named volume `pgdata`, which holds the data directory outside the container filesystem. The database container may be replaced or rebuilt without loss, and only `docker compose down -v` removes the volume.

Concurrency control

Two situations require explicit control: a decision taken on a value that another transaction may change before the write lands, and two requests competing to create the same uniquely identified row. Table 4 enumerates every instance of both in the system.

The first three cases share one structure: the application checks before writing, but the index is what decides. The check only improves the error message, for the reason given in Section 5.8.

Table 4: Concurrent conflicts arising in the system and the mechanism that resolves each.

Conflict	Origin	Resolving mechanism
Duplicate posting	Retransmission after a lost response, or two browser tabs	Partial unique index on (<code>OwnerId</code> , <code>IdempotencyKey</code>). The violation is caught as <code>SQLSTATE 23505</code> , the transaction is rolled back, and the committed row is returned.
Duplicate account name	Two concurrent create requests from one owner	Unique index on (<code>OwnerId</code> , <code>Name</code>), reported to the caller as 409.
Duplicate registration	Two concurrent sign ups	Unique indexes on <code>Username</code> and <code>Email</code> , reported as 409.
Overdraft through write skew	Two withdrawals from one Asset account	<code>FOR NO KEY UPDATE</code> on the credit row, held until commit. The second request waits and then reads the updated balance.
Lock cycle	Two transfers posted in opposite directions	The explicit mode is chosen so that the implicit foreign key lock does not conflict with it, so no wait-for cycle can form.
Exhaustion by password hashing	A burst of sign in or registration requests	Concurrency limiter on the bcrypt policy, which bounds simultaneous verifications and answers 429 beyond the queue limit.

6 Containerization

`docker compose up -build` starts three containers. The database becomes available first, the API waits until the database is accepting connections, and the Next.js server waits until the API is serving requests. Figure 7 states the topology and the startup dependencies.

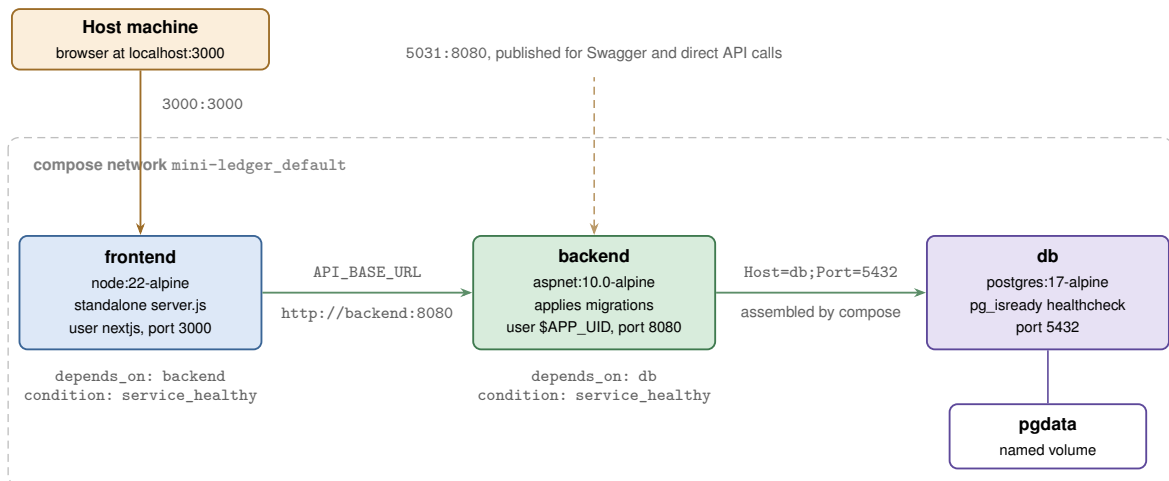


Figure 7: The three containers. Only the frontend needs to be reachable from the host; the backend port is published for convenience, not because the browser uses it.

6.1 Image construction

Both images are built in multiple stages. The API is compiled with `mcr.microsoft.com/dotnet/sdk:10.0-alpine` and shipped on `mcr.microsoft.com/dotnet/aspnet:10.0-alpine`. The runtime image carries the .NET runtime without a compiler, so neither source code nor build tooling is present in the published image.

The restore step is separated from the source copy so that the layer cache remains effective:

```

COPY backend.csproj .
RUN dotnet restore
COPY . .

```

```
RUN dotnet publish -c Release -o /app --no-restore
```

Modifying a `.cs` file invalidates only the final two instructions. The package download is reused until `backend.csproj` itself changes.

The Next.js image is built in three stages. `deps` executes `npm ci`, which installs precisely the lockfile and fails if the lockfile and the manifest disagree; `builder` executes `next build`; `runner` copies `.next/standalone` and `.next/static` only. The standalone output traces the files the server reaches at runtime and emits a minimal `node_modules`, so `typescript`, `tailwindcss`, `postcss` and `shadcn` are present at build time and absent at runtime.

Both runtime images execute as a non root user. The API uses the `$APP_UID` account defined by its base image, and the Next.js image creates `nextjs` with uid 1001. Neither process writes to the container filesystem, so neither is granted the ability to do so. This also determines the listening port: the API listens on 8080 rather than 80, because binding a port below 1024 requires a privilege that the non root account does not hold.

One instruction in the API image addresses a specific defect:

```
RUN apk add --no-cache krb5-libs
```

Npgsql probes for Kerberos authentication support when opening a connection. Alpine does not ship that library, so in its absence every startup emits a failure to load `libgssapi_krb5.so.2`. The connection succeeds regardless, but a spurious error line in the startup log is a worse outcome than the two megabytes the package occupies.

6.2 Service startup ordering

`depends_on` combined with `condition: service_healthy` performs a functional role rather than a decorative one. PostgreSQL starts, executes its initialisation scripts, and then restarts before it is genuinely ready. Waiting for the port alone would allow the API to connect during that interval and fail. `pg_isready` reports actual readiness.

The API declares its own healthcheck against `/health`, an anonymous endpoint that deliberately performs no database access. It answers a single question: whether this process is serving requests. `docker compose up -d` returns as soon as a container starts, which precedes Kestrel binding its port, so in the absence of this check the Next.js server could start against an API that is not yet accepting connections.

The API applies pending migrations at startup:

```
await using (var scope = app.Services.CreateAsyncScope())
{
    await scope.ServiceProvider.GetRequiredService<AppDbContext>().Database.MigrateAsync();
}
```

A container starts against an empty database, so the schema must be created before the first request arrives. The call is idempotent: against a database already at the latest migration it performs no work, which is what makes restarting safe.

6.3 Configuration and secrets

Secrets are supplied through a `.env` file excluded from version control. `.env.example` is the committed template and contains no real values. Compose performs the substitution and assembles the API connection string from the same variables used to create the database container, so the two cannot diverge:

```
ConnectionStrings__DefaultConnection:
    "Host=db;Port=5432;Database=${POSTGRES_DB};Username=${POSTGRES_USER};Password=${POSTGRES_PASSWORD}"
AppSettings__Token: ${JWT_SIGNING_KEY}
```

The double underscore is the .NET convention for mapping an environment variable onto a nested configuration key. `API_BASE_URL` is set to `http://backend:8080`, using the service name rather than `localhost`, because each container holds its own loopback address within the network. The value is read per request on the server, so one Next.js image runs unchanged in any environment.